Final Project Detailed Report

To-Do List Application

Berra Erkol
Student ID: 25120102007

Detailed Project Report: JavaFX To-Do List Application

1. Project Overview

Managing our daily tasks is very important in today's busy world. For my final project, I developed a desktop To-Do List application. I used the Java programming language and the JavaFX library for the visual interface (GUI).

My main goal was to build a real and useful application, not just a simple list. As the project rules require, my application has three main parts:

1. **Basic Functions:** Users can add new tasks, complete them, and delete them.
2. **Authentication System:** Users can register to the system, log in securely, and log out.
3. **File Processing:** The application saves all user data and tasks to the local computer automatically.

This application helps users organize their daily work safely and easily.

2. Special and Different Features

Many basic to-do list applications only have one list for everyone. I wanted to make my project better and different. Because of this, I added these special features:

* **Multi-User System:** Many different people can use this application on the same computer. Everyone registers with their own username and password. User A cannot see or change User B's tasks. The data is completely separated and safe.

* **Simple Text File (.txt) Storage:** I did not use a complex database (like SQL) or hard-to-read binary files. I used standard `.txt` text files to save data. The system saves passwords in a `users.txt` file. Also, it creates a special file for each user, like `Ahmet_tasks.txt`. This makes the files very easy to open, read, and check on any computer.

* **"Mark as Completed" Feature:** When a user finishes a task, they do not have to delete it immediately. They can select the task and click the "Complete" button. The application puts a checkmark `[✓]` at the beginning of the task. This helps users see what they have finished before they delete it forever.

3. Design and Coding Details

I designed the architecture of the application in three main parts: User Interface (UI), Event-Driven Code, and File Saving (File I/O).

3.1. User Interface (UI) Design

I used JavaFX to make a clean and modern screen. The application works in one main window (`Stage`), but it switches between two different screens (`Scene`):

* **The Login and Register Screen:** I used a `VBox` layout to put the text boxes and buttons from top to bottom. I put a `TextField` for the username and a `PasswordField` so the password is hidden with stars. I put everything in the center of the screen to make it look nice.

* **The Main Task Screen:** After the user logs in, they see the main application. I used an `ObservableList` and connected it to a `ListView`. This is very important. Thanks to `ObservableList`, when the user adds or deletes a task in the code, the screen updates automatically. I do not need to refresh the page manually. I put the input box and buttons side by side at the bottom using an `HBox` layout.

3.2. Event-Driven Programming

The application waits for the user to do something. This is called Event-Driven Programming. I used Java Lambda expressions (`e -> { ... }`) to write clean code for button clicks.

* **Button Actions:** Every button has a `.setOnAction()` method. For example, when the user clicks the "Register" button, the code checks if the text boxes are empty. If they are empty, it shows an error message.

* **Error Pop-ups:** To help the user, I created a special `showAlert()` method. If the user writes a wrong password or tries to delete a task without selecting it first, a small JavaFX warning window (Alert) opens and tells them what to do.

* **Closing the Window Safely:** What happens if the user clicks the red "X" button to close the app without logging out? To stop data loss, I added a `window.setOnCloseRequest(...)` method. When the user closes the window, this code runs and automatically saves the tasks to the `.txt` file before the program closes.

3.3. File Processing (Saving Data)

I used the `java.io` library to read and write files.

* **Fast Login Verification:** When the app opens, the `loadUsers()` method reads the `users.txt` file line by line using `BufferedReader`. It separates the username and password using a comma (`split(",")`). Then, it puts them into a `HashMap`. I used a `HashMap` because it can find a username and check the password instantly (O(1) time). It is very fast.

* **Saving Tasks:** When a user logs in, the app finds their personal `.txt` file and loads their tasks. When they log out, the `saveTasks()` method uses `BufferedWriter`. It takes all the tasks from the list and writes them line by line (`newLine()`) back to the text file.

4. Problems and Solutions

During this project, I had some problems, but I found good solutions:

1. **Problem - Changing Users:** When one user logged out and a new user logged in, the old tasks were still on the screen.

* *Solution:* I fixed this by completely resetting the screen. When a user clicks "Logout", my code makes `currentUser = null` and builds the login screen again from zero.

2. **Problem - Crashing on Delete:** If the user clicked the "Delete" button but did not select any task from the list, the application crashed.

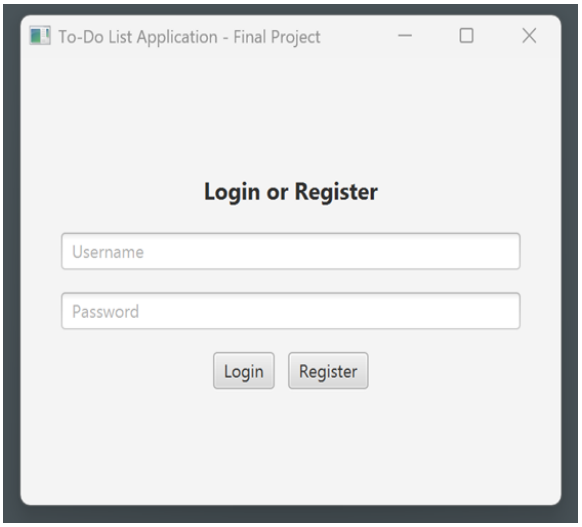
* *Solution:* I used `getSelectionModel().getSelectedIndex()`. I added an `if` condition. The code checks if the selected index is greater than or equal to 0 (`>= 0`). If the user does not select anything, it does not crash; it just shows a warning message.

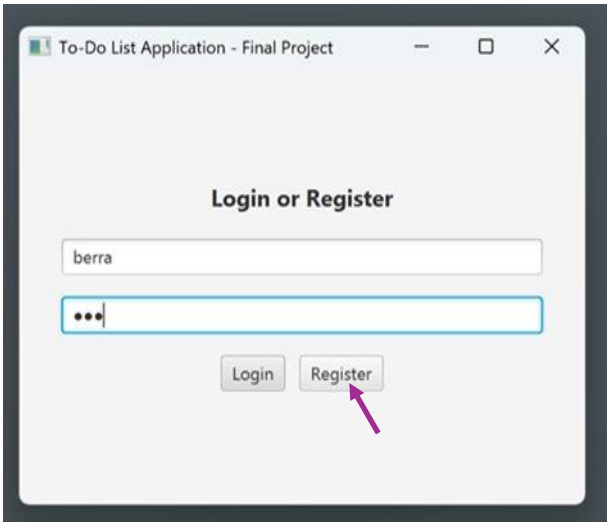
5. Visual Examples of the Application

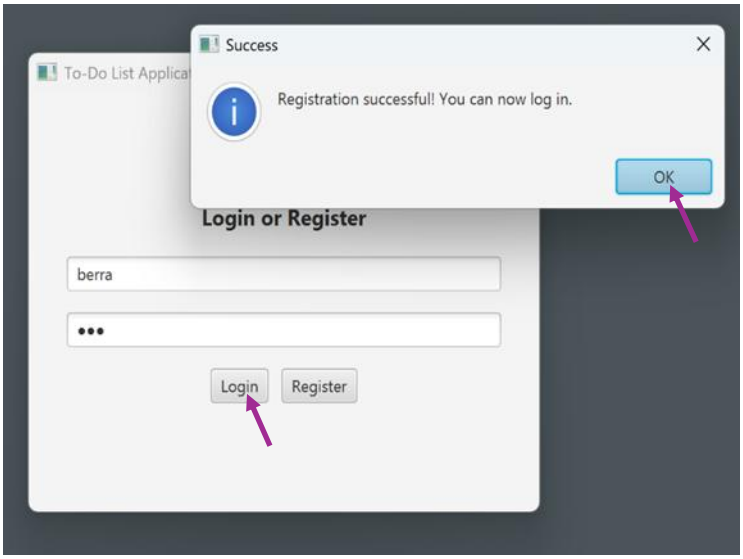
Below, you can see the pictures of my working application:

Step 1: User Registration and Login

When the application starts, you see the login screen. You can write a username and password and click "Register". The app saves your information.

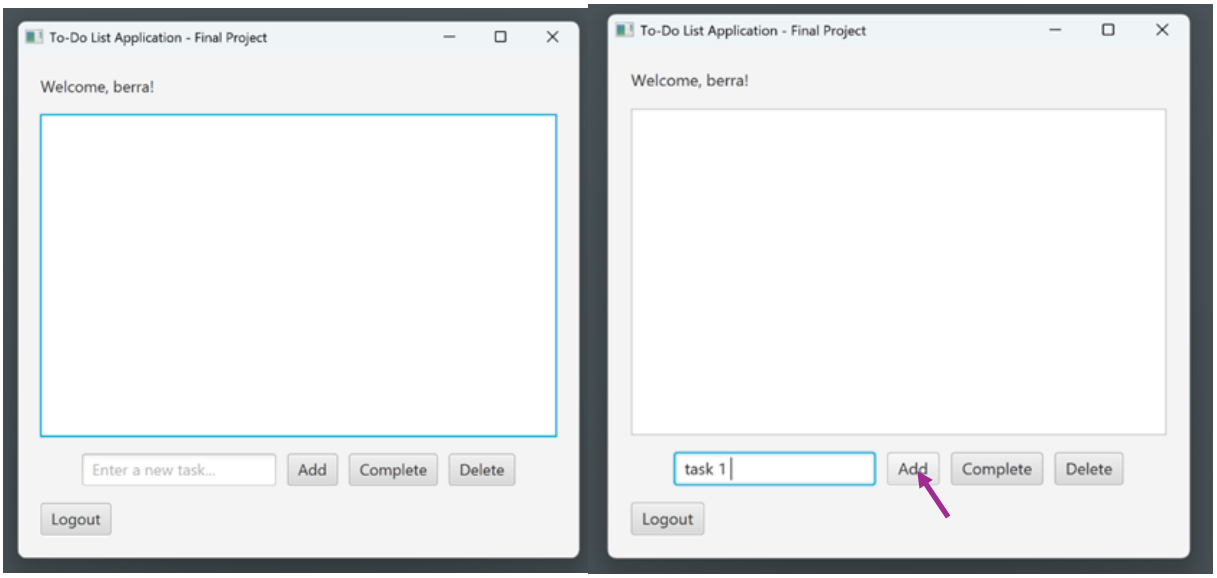


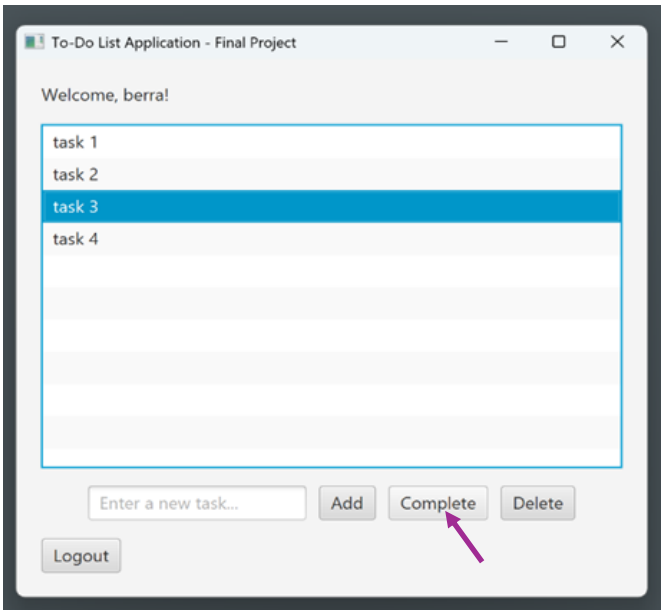




Step 2: Main Dashboard and Adding Tasks

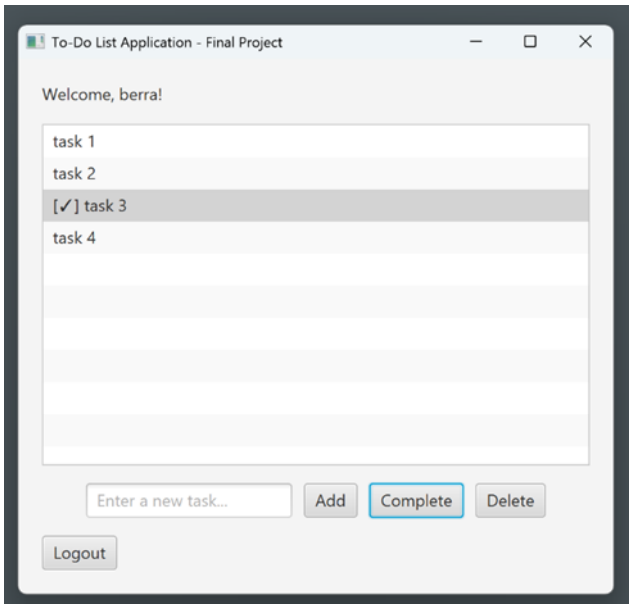
After logging in, you see your personal list. You can write your daily tasks in the text box and click "Add" to put them in the list.





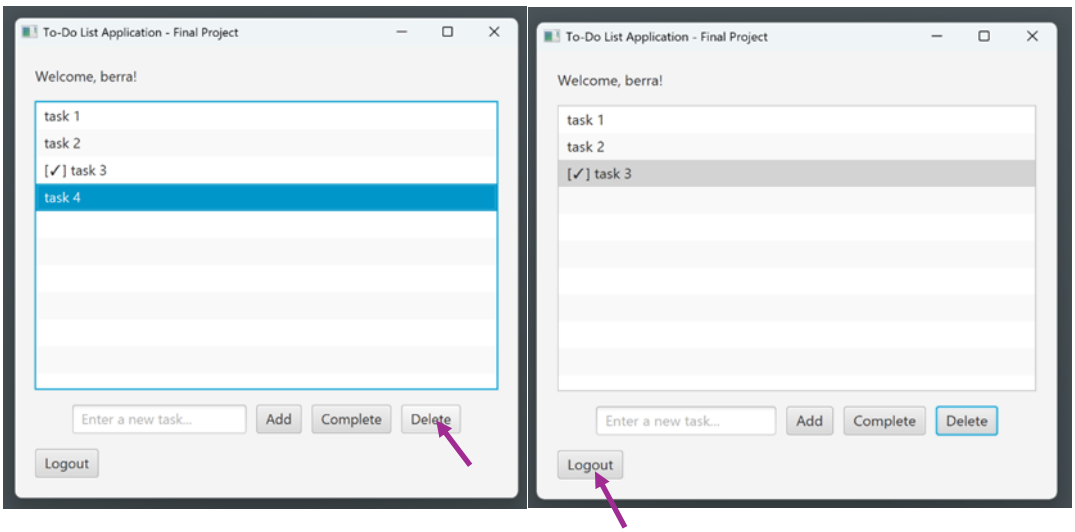
Step 3: Completing Tasks

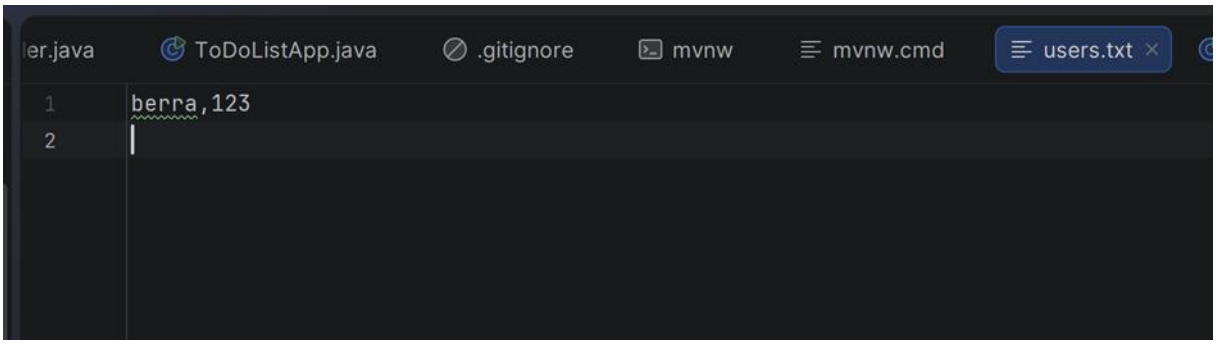
If you finish a task, you select it and click "Complete". The app adds a `[✓]` mark to the task so you know it is done.

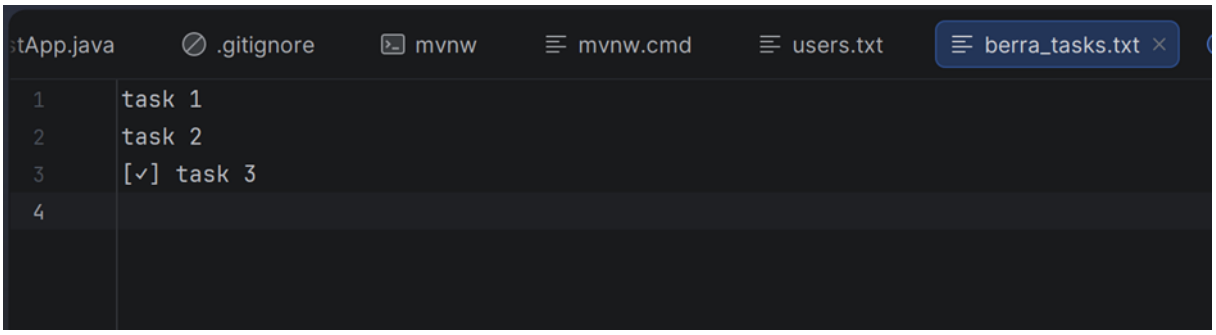


Step 4: Deleting Tasks and Local Files

You can select any task and click "Delete" to remove it. When you click "Logout", the application saves everything to the folder. You can see the `.txt` files in the computer folder below.







6. Conclusion

In conclusion, I successfully used Object-Oriented Programming (OOP) and JavaFX to build this project. I added a secure login system and used local text files to save data. The application works perfectly, follows all the project rules, and is very easy to use for daily task management.

GitHub Link:

https://github.com/berraerkol/ToDoListApp-JavaFX-Final